Programmation Modulaire en Langage C

Par Robert DIASSÉ

COURS C —Introduction

1. Introduction : qu'est-ce que le langage C ?

Le **langage C** est un langage de programmation inventé dans les années 1970.
Il sert à écrire :

- des systèmes (Linux, Windows, macOS → beaucoup de parties écrites en C)
- des logiciels rapides
- des programmes embarqués (robots, cartes électroniques…)
- des applications nécessitant exactitude et performance.

Le C est un **langage compilé** :

- On écrit un fichier texte appelé **fichier source** : `programme.c`.
- Ce fichier est traduit (compilé) en un **programme exécutable** que l'ordinateur peut comprendre.

On peut imaginer :

> *Le fichier .c est la recette → le compilateur est le cuisinier → le programme final est le plat.*

---

2. Comment créer un fichier C ?

Pour créer un programme C :

1. Ouvrir VS Code / CodeBlocks / un éditeur simple.
2. Créer un fichier :
   `monpremier.c`
3. Écrire du code C dedans.

---

3. Structure minimale d'un programme C

Voici **le plus petit programme valide en C** :

```c
#include <stdio.h>

int main(void) {
    return 0;
}
```

Il ne fait rien, mais il est **correct**.
Voyons chaque ligne :

`#include <stdio.h>`

- Ceci n'est pas du "C", mais du **préprocesseur**.
- Cela signifie :

  > *"Inclure les déclarations permettant d'utiliser les entrées/sorties standard (affichage, lecture, etc.)"*

- `stdio.h` signifie **Standard Input Output**.
- Sans cette ligne, on ne peut pas utiliser `printf` ou `scanf`.

`int main(void) { ... }`

C'est la **fonction principale** du programme.

- `int` = la fonction renvoie un entier (généralement 0 si tout se passe bien).
- `main` = nom obligatoire.
- `(void)` = la fonction ne reçoit aucun paramètre.
- `{ ... }` = bloc d'instructions.

`return 0;`

- Cela signifie : **le programme s'est bien terminé**.
- Le système d'exploitation reçoit cette information.

Conclusion :
Tout programme C commence par `main`, même les plus grands logiciels.

---

4. Comment afficher un texte ? (Première fonction : printf)

Pour afficher du texte, on utilise :

```
printf("Bonjour !");
```

Mais cette fonction vient de la bibliothèque `stdio.h`, donc il faut **obligatoirement** écrire :

```
#include <stdio.h>
```

Exemple complet :

```
#include <stdio.h>

int main(void) {
    printf("Bonjour C !\n");
    return 0;
}
```

---

5. Les chaînes de caractères : texte entre guillemets `" ... "`

Lorsque vous écrivez :

```
"Bonjour C !"
```

C'est ce qu'on appelle une **chaîne de caractères** (type `char[]` en interne).
En C, **le texte doit être entouré de guillemets doubles** `" "`.

Exemple incorrect :
`Bonjour` ✕

Exemple correct :
`"Bonjour"` ✔

---

6. Les codes d'échappement ou de contrôle (sauts de ligne, tabulation...)

Dans un texte, on peut insérer des **commandes spéciales** à l'intérieur de `printf`.

Code	Signification	Effet

Code	Signification	Effet
`\n`	Nouvelle ligne	Descend à la ligne suivante
`\t`	Tabulation	Ajoute un espace horizontal large
`\"`	Guillemet	Affiche "
`\\`	Antislash	Affiche \

Exemple :

```
printf("Bonjour\nComment va ?\n");
printf("Nom\tAge\tVille\n");
```

Affichage :

```
Bonjour
Comment va ?
Nom     Age     Ville
```

---

7. Les commentaires (explications dans le code)

Deux formes possibles :

```
// Commentaire court

/* Commentaire
   sur plusieurs lignes */
```

Les commentaires ne sont **pas exécutés**, ils servent à expliquer.

---

8. Les variables : qu'est-ce que c'est ?

Une **variable** est une petite zone de la mémoire de l'ordinateur qui contient une valeur.

On lui donne :

- un **nom**
- un **type**
- une **valeur**

Exemple :

```
int age = 20;
```

Signifie :

- **type :** `int` → entier
- **nom :** `age`
- **valeur :** 20
- **mémoire occupée :** dépend de la machine, souvent 4 octets (32 bits)

Les règles pour nommer une variable

- Doit commencer par une lettre ou `_`
- Pas d'espace
- Sensible à la casse (`Nom` ≠ `nom`)
- Doit être claire (`note`, `prix`, `age`...)

---

9. Les types en C + leurs tailles mémoire

Type	Signification	Taille approximative
`char`	caractère	1 octet
`int`	entier	2 ou 4 octets
`float`	nombre réel simple précision	4 octets
`double`	nombre réel double précision	8 octets

Vous pouvez demander la taille exacte :

```
printf("%zu\n", sizeof(int));
```

---

10. Les constantes : valeurs qui ne changent pas

Deux façons :

1) Constante du préprocesseur :

```
#define PI 3.14
```

C'est remplacé **avant** la compilation.

2) Constante C :

```
const int MAX = 100;
```

On ne peut pas modifier `MAX`.

---

11. Entrée utilisateur : scanf

`scanf` permet de **lire une valeur entrée par l'utilisateur**.

Exemple :

```
int age;
printf("Votre âge : ");
scanf("%d", &age);
```

Pourquoi `&age` ?

L'opérateur `&` signifie **adresse de la variable**.
`scanf` doit connaître **où stocker la valeur**.

---

12. Exemple complet et très simple

```
#include <stdio.h>

int main(void) {
    int age;
    float note;
    char prenom[20];

    printf("Votre prenom : ");
    scanf("%19s", prenom);

    printf("Age : ");
    scanf("%d", &age);

    printf("Note : ");
    scanf("%f", &note);

    printf("\n--- Résultat ---\n");
    printf("Prenom : %s\n", prenom);
    printf("Age : %d\n", age);
    printf("Note : %.2f\n", note);
```

```
    return 0;
}
```

Tout ici est maîtrisable sans structures de contrôle.

---

13. Exercices

Exercice 1 : Afficher un texte

Écrire un programme affichant :

```
Bonjour !
Je découvre le langage C !
```

Exercice 2 : Lire et afficher une variable

Lire un entier `x` et afficher :

```
Vous avez saisi : x
```

Exercice 3 : Lire nom, âge, note

Lire un prénom (sans espace), un âge et une note, puis afficher ce résumé.

Exercice 4 : Tester les codes d'échappement

Afficher un tableau aligné comme :

```
Nom     Age     Ville
Ali     20      Dakar
```